

Deep Research on Entity["organization","Awwwards","web design awards platform"] Sites and How to Build One Like Them

What makes Awwwards sites feel “too good”

Awwwards is not judging “a nice website.” It’s explicitly scoring *talent and effort* using weighted criteria: **Design (40%)**, **UX/UI (30%)**, **Creativity (20%)**, and **Content (10%)**. ¹ That weighting alone explains a lot of the vibe: they’re optimized to *feel* exceptional before they’re optimized to be purely informational.

The evaluation framework also hard-bakes an expectation of craft across the classic four pillars—Design, Creativity, Usability, Content—because those are the official “key criteria” Awwwards commits to for site evaluation. ² When you combine that rubric with a jury-based process (experienced designers/devs) you get a predictable outcome: sites that treat interaction, motion, and narrative as first-class design materials, not decoration. ¹

A small but revealing signal: Awwwards organizes inspiration browsing “by technology” (not just by industry or style), with buckets like **WebGL**, **3D**, **React**, **GSAP**, **Webflow**, **Framer**, **Shopify**, **WordPress**, and **CSS animations**. ³ That taxonomy is basically Awwwards admitting: “Yes, the tech stack is part of the aesthetic.”

Interaction patterns that show up again and again

Awwwards-winning experiences often use interaction as *story structure*: the page isn’t just scrolled; it’s *performed*. You see this explicitly in Awwwards case studies describing “continuous scroll narrative” experiences designed to provoke curiosity and sustain engagement. ⁴

A few recurring interaction patterns (with receipts):

- **Scroll as choreography (not navigation):** Awwwards’ own parallax explainer describes the effect as multiple background layers moving at different speeds to create perceived depth (a faux-3D effect) during scrolling. ⁵ That’s the conceptual backbone of a huge chunk of “wow” sites.
- **Micro-interactions + transitions as “feel”:** The Rich Brown portfolio case study frames an immersive experience as being “brought to life” via micro-interactions and page/screen transitions, while keeping the underlying structure simple. ⁶
- **Motion tech diversity:** Awwwards’ animation inspiration section explicitly points to the modern palette: CSS3, Canvas, SVG, WebGL, plus the broader ecosystem of animation techniques and APIs.

⁷

- **Sound as depth (used sparingly):** In the KODE Immersive case study, sound is treated as an atmospheric layer (the “80/20 rule”) with hover/transition SFX to amplify immersion without overwhelming. ⁴

That “interactivity” is rarely one trick. It’s stacking: scroll + cursor + timing + transitions + subtle physics + audio + typographic motion, all synchronized so it feels like a single instrument.

The technology stack behind the “impossibly interactive” look

There isn’t one stack, but there are very common *stack shapes*. Awwwards’ own browsing categories highlight recurring choices (WebGL/3D, GSAP, React, Webflow/Framer, etc.). ³ The case studies show how those ingredients get combined.

WebGL and 3D as the “background engine”

When a site uses true 3D (not just 2D parallax), it’s usually WebGL-driven. WebGL is a JavaScript API for high-performance 2D/3D rendering in a `<canvas>` without plugins, leveraging hardware acceleration. ⁸ Awwwards case studies repeatedly frame “Three.js + WebGL” as the storytelling backbone for immersive brand experiences. ⁹

Because smoothness is the religion here, animation loops commonly rely on `requestAnimationFrame()`, which schedules work before the next repaint for smoother animation than timer loops. ¹⁰ (The Shapefarm case study explicitly mentions combining Lerp with `requestAnimationFrame` to achieve smoothness. ¹¹)

Animation orchestration (GSAP and friends)

In the Rich Brown case study, animations use GSAP alongside SmoothState.js for AJAX-style transitions, and even inline SVG masking effects are animated with GSAP because it can drive individual SVG elements smoothly across devices. ⁶ On the Shapefarm project, GSAP is described as the primary animation library across Canvas and Three.js scenes. ¹¹

This is the “orchestra conductor” role: not just animating one element, but synchronizing many timelines with scroll/cursor state. A key enabler for that modern pattern is scroll-triggered animation tooling; for example, GSAP’s ScrollTrigger is explicitly designed for scroll-based animations (scrub/pin/snap/trigger). ¹²

Performance architecture: off-main-thread rendering and batching

The KODE Immersive case study calls out a very “Awwwards” move: moving expensive rendering to **OffscreenCanvas** and running the logic inside a **Web Worker** to improve responsiveness. ⁴ That aligns with MDN: OffscreenCanvas decouples canvas rendering from the DOM and can run rendering operations in a worker context to avoid heavy main-thread work. ¹³ Web Workers are the standard way to run scripts in a background thread so the UI thread stays responsive. ¹⁴

Also in KODE: a key optimization is reducing draw calls by batching meshes that share materials (dropping from “40 or 50” calls to “one” in their example). ⁴ This matches common 3D performance guidance: fewer

draw calls generally helps because per-call overhead and state changes can bottleneck the CPU/GPU pipeline. ¹⁵ In Three.js specifically, InstancedMesh is documented as a way to render many objects with shared geometry/material but different transforms, explicitly to reduce draw calls and improve performance. ¹⁶

Site architecture: SSG, headless CMS, CDN, lazy loading

The Shapefarm technical case study reads like a checklist of modern “fast but fancy” architecture:

- Static site generation with a framework (they chose Nuxt as SSG). ¹⁷
- Headless CMS via the WordPress REST API (client edits content in WordPress; the front-end consumes JSON). ¹⁸
- A Jamstack-style “decoupled” model, which Jamstack.org describes as an architecture designed to make the web faster, more secure, and easier to scale via pre-rendering and decoupling. ¹⁹
- CDN distribution (S3 + CloudFront mentioned) plus asset slimming (WebP images, glTF models). ¹¹
- Lazy loading using the Intersection Observer API. ²⁰

That last one matters: Intersection Observer is commonly used for lazy-loading content as the page scrolls because it can observe visibility changes asynchronously. ²¹

How Awwwards-level sites are actually built

The myth is “a genius designer made a cool layout.” The reality is closer to “a studio ran a small R&D project and packaged it as a website.”

The KODE Immersive case study lays out a deliberate, phased process: discovery (strategy + style tiles), create (prototype-driven feasibility loops), craft (shader logic, ambient lighting, cursor haptics), then deliver (stress-testing performance across browsers/devices/speeds). ⁴ That’s not fluff—those steps map directly to the technical complexity of WebGL-heavy sites.

Shapefarm’s case study shows the same pattern in a more production-y form:

- Start with mood boards (they used Dribbble, “company”, “Pinterest”, “visual bookmarking platform”) to lock taste before execution. ¹¹
- Build internal tooling when needed (they created a Three.js tool to export 3D parts as PNGs at any angle/size to speed design iteration). ¹¹
- Engineer for loading constraints (optimize assets; use CDN; lazy load; reduce library size; make certain canvas animations disappear post-load so they don’t tax runtime). ¹¹

Even the older Rich Brown case study shows the “craft stack” pattern: build templates from scratch, use tooling for fast iteration (SASS + Gulp + BrowserSync), then implement signature transitions (SVG masking, smooth page transitions) and deploy as static files. ⁶

So the honest recipe is: 1) narrative + interaction intent,
2) a prototype that proves the interaction,
3) an asset pipeline that supports the visuals,

- 4) a performance plan that prevents the whole thing from becoming a slideshow,
- 5) QA that treats devices/browsers as hostile environments.

Blueprint for your interactive “distraction → focus” side-scrolling site

You described a very specific cinematic concept:

- Typewriter intro: “are you ever just trying to get work done?”
- Pop-up ads escalate, fill screen, then float away
- Type: “.....but keep getting distracted”
- Sell a focused study platform
- Horizontal / side-scrolling sections that pitch value
- Background: infinitely long grid; stacked 2D grid planes along Z for parallax illusion
- Smoke + ambience vibes
- Buttons with depth + glow
- Typography: professional, not generic-modern; like Domaine Display or Abril Fatface

Below is a build plan that matches how Awwwards-class sites structure these experiences, using techniques directly documented in the case studies and platform docs above.

Recommended architecture: WebGL background + DOM foreground

Core idea: Render the infinite grid + smoke + subtle post FX in a WebGL `<canvas>`, and render text, UI, pop-ups, and clickable CTAs in normal DOM. This split is common because: - WebGL is ideal for “infinite world” visuals and parallax depth. ⁸

- DOM is better for accessibility, crisp text, selection, and UI semantics. ²²

- Moving heavy render work off the main thread is possible if needed via OffscreenCanvas + Worker. ²³

Horizontal scrolling: two solid implementation options

Option A: Native horizontal scroll (cleanest + most accessible) Use a horizontally scrolling container with CSS scroll snapping, which MDN documents as a module for defining snap positions for controlled “paging” behavior. ²⁴

This naturally supports trackpads, keyboards (with some help), and doesn’t require faking scroll physics.

Option B: Vertical scroll that drives horizontal motion (the “Awwwards classic”) Pin a section and translate the horizontal panels based on scroll progress, typically using a scroll animation system. ScrollTrigger is explicitly built to create scroll-based animations including pinning and scrubbing. ¹²

This pattern is very good for cinematic pacing (you can “hold” a scene until something finishes happening).

Typewriter effect: CSS steps() is the simplest reliable approach

MDN documents the `steps()` timing function as dividing time into discrete intervals—exactly what you want when revealing characters one “tick” at a time. ²⁵ You implement the reveal with a standard

`@keyframes` animation, which MDN documents as the rule defining the keyframe sequence for CSS animations. ²⁶

A minimal pattern (conceptually): - Put the line in a fixed-width container - Animate `width` from `0ch` to `Nch` - Use `steps(N)` so it jumps one character per step

Because intense motion is part of your concept, also build a “reduced motion” path. `prefers-reduced-motion` exists specifically to detect when a user wants less non-essential motion. ²⁷

Pop-up ad storm: build it like a controlled VFX sequence (not like malware)

Your goal is to *simulate* distraction as a narrative device, so treat pop-ups as choreographed UI sprites:

- Generate ad cards as DOM nodes (position: fixed)
- Animate them with **transform + opacity** for performance, because animating those properties avoids layout/paint and can stay compositor-only. ²⁸
- Escalate in three phases:
 - Phase 1: 1–3 popups, easy to close
 - Phase 2: 6–12 popups, overlapping, jitter, “close” moves
 - Phase 3: full-screen takeover, then a “whoosh away” moment into calm

Why this works psychologically (and why it’s risky outside an art piece): - Popups interrupt tasks and are widely perceived as intrusive; NN/g explicitly warns against prompting during critical tasks and notes users often reflex-close to return to their goal. ²⁹

- Research on notification-caused interruptions finds reducing interruptions benefits performance and reduces strain in a field experiment setting (i.e., interruptions have real cost). ³⁰

So your narrative has teeth: you’re dramatizing a real phenomenon, then selling relief.

One practical warning: if this page is meant to rank in search or behave like a normal marketing site, intrusive interstitial behavior can be a problem. Google’s guidance defines intrusive interstitials/dialogs as elements that obstruct the user’s view of content, and recommends avoiding them. ³¹

Solution: make this sequence *opt-in* (e.g., “Start experience”), or treat it as an interactive demo page separate from SEO-critical landing pages.

Infinite parallax grid background: how to build it (and keep it fast)

You described: “infinitely long grid” + “2D grids stacked along the z-axis to give illusion of parallax when side scroll.”

That’s basically: - A camera moving along the X axis (side travel) - Multiple grid planes at different Z depths - Slightly different scroll-to-motion ratios per plane (parallax) - Fog/smoke to soften the distance and hide looping seams (cheat code for infinity)

In Three.js terms, the tools you'll likely use (all directly documented):

- **Fog / haze:** Use Fog or FogExp2 for atmosphere. FogExp2 is explicitly exponential-squared, giving clearer near view and denser far fog. ³²
- **Instancing for grid segments:** Use InstancedMesh for repeated geometry to reduce draw calls and improve performance. ¹⁶
- **Post-processing pipeline:** EffectComposer manages a chain of post-processing passes. ³³
- **Glow:** UnrealBloomPass provides bloom with a mip chain + multi-radius blur for “good quality and performance” (relative to naive approaches). ³⁴
- **Grain / ambience:** FilmPass can create a film grain effect. ³⁵

Looping trick (to make “infinite”): Maintain a set of grid segments that cover (say) 3–5 screen widths. As the camera moves, wrap segments from left to right (modulus arithmetic). Fog helps hide the moment you recycle geometry. ³⁶

Sync movement with scroll smoothly: Use a “target position” driven by scroll progress, and a “current position” that lerps toward the target in your animation loop. This exact smoothness approach (lerp + requestAnimationFrame) is called out in the Shapefarm case study. ³⁷

If the background gets heavy: Offload the canvas into a worker using OffscreenCanvas (as KODE Immersive did for resource-intensive canvases). ³⁸

Smoke, ambience, and “floating depth” buttons

You want “smoke and ambience vibes” plus buttons that feel like they float with glow.

Visually, you can combine: - **3D fog** (depth cue) ³² - **film grain** (texture cue) ³⁵ - **bloom** on bright accents and subtle UI glows ³⁴ - **DOM button styling:** soft shadow + slight translateZ/transform on hover + glow pseudo-element - Animate button hover/press states with transform/opacity (compositor-friendly). ²⁸

Typography: Domaine Display vs Abril Fatface (and licensing reality)

You name-dropped two good references, and they're meaningfully different:

- **Domaine Display** is positioned by Identity[“company”, “Klim Type Foundry”, “type foundry wellington nz”] as a refined serif for large sizes. ³⁹ Klim's licensing model describes one-off fees and no expiry across license types (desktop/web/app/etc.). ⁴⁰
Practical implication: gorgeous, premium, but you budget for webfont licensing.
- **Abril Fatface** is part of the broader Abril system; it's widely distributed under the SIL Open Font License (OFL) (for the Fatface cut). ⁴¹
Practical implication: easy to ship, good editorial punch, less “boutique” than Domaine but still distinctive.

If you want “professional but not generic-modern,” your best move is: - Use **one** strong display face (Domaine Display or Abril Fatface) - Pair with a text face that's quieter and more readable (editorial serif or restrained sans) - Keep motion typographic (kerning, small shifts, reveal timing), not “techno.”

Suggested narrative structure and copy tone

You said: professional, not overly modern/generic. So avoid “supercharge your productivity” cliché copy. Aim for “editorial clarity.”

A tight structure that matches your concept:

Scene 1: Intent

Typewriter: “Are you ever just trying to get work done?”

Subhead (quick fade-in): “No hacks. No hype. Just the reality of modern attention.”

Scene 2: Distraction event

Popups escalate. Audio tick rises. Cursor feels “sticky” for a moment.

Then everything exhales—popups drift away, fog deepens.

Typewriter: “.....but keep getting distracted.”

Scene 3: Relief and proposition

Headline: “Build a study environment that doesn’t fight you.”

Subhead: “A calm, structured space for focused work—designed to stay out of your way.”

Side-scroll panels (value)

Panel themes: - “Focus modes that actually reduce interruptions” - “A workspace built for long sessions” - “Tools that support momentum, not novelty” - CTA: “Try a session” / “See how it works”

This bridges nicely into real evidence about interruptions hurting performance/strain. ³⁰

Performance, accessibility, and “don’t accidentally sabotage yourself”

Awwwards sites often walk a knife edge: impressive *and* fragile. If you want this to feel premium instead of janky, you need guardrails.

Keep motion performant

Web.dev’s guidance is blunt: for best animation performance, stick to **transform and opacity** changes to avoid layout/paint and rely on compositing. ²⁸ This matters directly for your popup storm (lots of moving layers).

If you must run a continuous loop (WebGL scene, shader noise, parallax), use `requestAnimationFrame()`—it schedules animation updates before the next repaint. ¹⁰

Respect reduced motion (and reduced chaos)

If someone has `prefers-reduced-motion: reduce`, your experience should: - Skip the popup storm (or dramatically reduce it) - Replace typewriter animation with an instant reveal - Reduce camera motion/

parallax amplitude

This media feature exists precisely to support users who prefer fewer non-essential animations. ²⁷

Be careful with “intrusive interstitials” if this is a real marketing page

Your concept intentionally uses intrusive overlays. That’s fine as art—but: - NN/g documents that premature popups (before users get value) are highly intrusive and usually ignored/closed immediately. ²⁹ - Google’s documentation on intrusive interstitials frames them as obstructions to content access and recommends avoiding them. ³¹

If you want both the cinematic effect *and* a sane business outcome, the clean compromise is: - Make the “distraction storm” an **opt-in** interactive demo - Put your real conversion flow on a calmer page (or after the demo resolves)

If the WebGL background gets heavy, steal from the best

KODE Immersive explicitly reports performance gains by moving heavy canvas rendering to OffscreenCanvas inside a Web Worker. ³⁸ That’s not always necessary, but it’s a proven technique when the main thread starts choking on animation + DOM + scroll listeners.

And if your “infinite grid” becomes many objects, use instancing: Three.js documents InstancedMesh as a way to reduce draw calls and improve overall rendering performance. ¹⁶

1 Evaluation System

https://www.awwwards.com/about-evaluation/?utm_source=chatgpt.com

2 Legal Terms

https://www.awwwards.com/terms/?utm_source=chatgpt.com

3 Sites Of The Day

https://www.awwwards.com/websites/sites_of_the_day/?utm_source=chatgpt.com

4 ⁹ ³⁸ Case Study: KODE Immersive

<https://www.awwwards.com/case-study-kode-immersive.html>

5 30 Great Websites with Parallax Scrolling

<https://www.awwwards.com/30-great-websites-with-parallax-scrolling.html>

6 Case Study: Rich Brown Portfolio

<https://www.awwwards.com/case-study-rich-brown-portfolio.html>

7 Best Animation Websites | Web Design Inspiration

https://www.awwwards.com/websites/animation/?utm_source=chatgpt.com

8 WebGL: 2D and 3D graphics for the web - Web APIs | MDN

https://developer.mozilla.org/en-US/docs/Web/API/WebGL_API?utm_source=chatgpt.com

10 Window: requestAnimationFrame() method - Web APIs - MDN

https://developer.mozilla.org/en-US/docs/Web/API/Window/requestAnimationFrame?utm_source=chatgpt.com

11 17 18 19 20 37 **Technical Case Study: Shapefarm Corporate Website**

<https://www.awwwards.com/technical-case-study-shapefarm-corporate-website.html>

12 **ScrollTrigger | GSAP | Docs & Learning**

https://gsap.com/docs/v3/Plugins/ScrollTrigger/?utm_source=chatgpt.com

13 23 **OffscreenCanvas - Web APIs - MDN**

https://developer.mozilla.org/en-US/docs/Web/API/OffscreenCanvas?utm_source=chatgpt.com

14 **Web Workers API - MDN - Mozilla**

https://developer.mozilla.org/en-US/docs/Web/API/Web_Workers_API?utm_source=chatgpt.com

15 **why are draw calls expensive?**

https://stackoverflow.com/questions/4853856/why-are-draw-calls-expensive?utm_source=chatgpt.com

16 **InstancedMesh – three.js docs**

https://threejs.org/docs/pages/InstancedMesh.html?utm_source=chatgpt.com

21 **Intersection Observer API - Web APIs | MDN - Mozilla**

https://developer.mozilla.org/en-US/docs/Web/API/Intersection_Observer_API?utm_source=chatgpt.com

22 **Web performance guides - MDN - Mozilla**

https://developer.mozilla.org/en-US/docs/Web/Performance/Guides?utm_source=chatgpt.com

24 **CSS scroll snap - MDN Web Docs**

https://developer.mozilla.org/en-US/docs/Web/CSS/Guides/Scroll_snap?utm_source=chatgpt.com

25 **animation-timing-function - CSS | MDN - Mozilla**

https://developer.mozilla.org/en-US/docs/Web/CSS/Reference/Properties/animation-timing-function?utm_source=chatgpt.com

26 **keyframes - CSS - MDN Web Docs**

https://developer.mozilla.org/en-US/docs/Web/CSS/Reference/At-rules/%40keyframes?utm_source=chatgpt.com

27 **prefers-reduced-motion - CSS - MDN Web Docs**

https://developer.mozilla.org/en-US/docs/Web/CSS/Reference/At-rules/%40media/prefers-reduced-motion?utm_source=chatgpt.com

28 **Stick to Compositor-Only Properties and Manage Layer ...**

https://web.dev/articles/stick-to-compositor-only-properties-and-manage-layer-count?utm_source=chatgpt.com

29 **Popups: 10 Problematic Trends and Alternatives**

https://www.nngroup.com/articles/popups/?utm_source=chatgpt.com

30 **Effects of task interruptions caused by notifications from ... - PMC**

https://pmc.ncbi.nlm.nih.gov/articles/PMC10244611/?utm_source=chatgpt.com

31 **Avoid intrusive interstitials and dialogs**

https://developers.google.com/search/docs/appearance/avoid-intrusive-interstitials?utm_source=chatgpt.com

32 36 **FogExp2 – three.js docs**

https://threejs.org/docs/pages/FogExp2.html?utm_source=chatgpt.com

33 **EffectComposer – three.js docs**

https://threejs.org/docs/pages/EffectComposer.html?utm_source=chatgpt.com

34 **UnrealBloomPass – three.js docs**

https://threejs.org/docs/pages/UnrealBloomPass.html?utm_source=chatgpt.com

35 **FilmPass – three.js docs**

https://threejs.org/docs/pages/FilmPass.html?utm_source=chatgpt.com

39 **Domaine Display fonts**

https://klim.co.nz/fonts/domaine-display/?srsId=AfmBOorF7xIXEg1hQFwF-DVAUU2IVHsIrhYoBh1B-hTM7ju-G7kZ6IIC&utm_source=chatgpt.com

40 **Font licences**

https://klim.co.nz/licences/?srsId=AfmBOoqA-rG5rR5_ttSUjETPMYo72HemteL6XkxmndoV8jsXrNXICoAC&utm_source=chatgpt.com

41 **License for font family 'Abril Fatface'**

https://www.fontsquirrel.com/license/abril-fatface?utm_source=chatgpt.com